

ISTEC — Instituto Superior de Tecnologias Avançadas do Porto  
CTeSP em Redes e Sistemas Informáticos · 1.º Ano · 2.º Semestre · 2025/2026  
**Unidade Curricular: PETD (Programação Estruturada e Tipos de Dados)**

## TRABALHO PRÁTICO FINAL

# Orçamento Familiar

*Sistema de Gestão Financeira para Famílias Portuguesas*

---

### Autores

Gonçalo F. · Tiago F. · Alexandre T.

### Professor

Prof. Mário Amorim

*Porto · Maio de 2026*

# Índice

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>1. Introdução.....</b>                             | <b>3</b>  |
| 1.1. Motivação.....                                   | 3         |
| 1.2. Objetivos do trabalho.....                       | 3         |
| <b>2. Análise de Requisitos.....</b>                  | <b>3</b>  |
| 2.1. Requisitos funcionais.....                       | 4         |
| 2.2. Requisitos não funcionais.....                   | 4         |
| 2.3. Tema escolhido.....                              | 4         |
| <b>3. Arquitetura da Solução.....</b>                 | <b>4</b>  |
| 3.1. Estrutura de diretórios.....                     | 5         |
| 3.2. Fluxo de dados.....                              | 5         |
| 3.3. Responsabilidades por módulo.....                | 5         |
| <b>4. Modelo de Dados.....</b>                        | <b>5</b>  |
| 4.1. Esquema geral.....                               | 5         |
| 4.2. Estrutura de uma receita.....                    | 6         |
| 4.3. Estrutura de uma despesa.....                    | 6         |
| 4.4. Categorias adaptadas à realidade portuguesa..... | 6         |
| 4.5. Decisões de design.....                          | 7         |
| <b>5. Implementação.....</b>                          | <b>7</b>  |
| 5.1. API do módulo logic.py.....                      | 7         |
| 5.2. Função obrigatória — calcular_media_mensal.....  | 8         |
| 5.3. Interface — main.py.....                         | 9         |
| <b>6. Validações e Tratamento de Erros.....</b>       | <b>9</b>  |
| 6.1. Regras aplicadas.....                            | 9         |
| 6.2. Tratamento de erros na interface.....            | 9         |
| <b>7. Execução e Demonstração.....</b>                | <b>10</b> |
| 7.1. Comando de execução.....                         | 10        |
| 7.2. Cenário de demonstração.....                     | 10        |
| <b>8. Distribuição de Tarefas.....</b>                | <b>10</b> |
| 8.1. Convenções de equipa.....                        | 11        |
| <b>9. Reflexão Crítica e Trabalho Futuro.....</b>     | <b>11</b> |
| 9.1. O que correu bem.....                            | 11        |
| 9.2. Limitações conscientes.....                      | 11        |
| 9.3. Possíveis evoluções.....                         | 11        |
| 9.4. Aprendizagens.....                               | 12        |
| <b>10. Conclusão.....</b>                             | <b>12</b> |
| <b>Referências Bibliográficas.....</b>                | <b>12</b> |

## 1. Introdução

O presente relatório descreve o desenvolvimento do trabalho prático final da unidade curricular de Programação Estruturada e Tipos de Dados (PETD), do 1.º ano do CTeSP em Redes e Sistemas Informáticos do ISTECS Porto, no ano letivo 2025/2026, sob orientação do professor Mário Amorim.

O projeto consiste numa aplicação de linha de comandos, escrita em Python 3, que permite a uma família registar e organizar as suas receitas e despesas mensais. Os dados são guardados em formato JSON e o programa oferece operações de inserção, consulta, edição, remoção e cálculo de saldos e médias.

### 1.1. Motivação

A gestão financeira doméstica é um problema transversal a qualquer agregado familiar e, em Portugal, a recente subida do custo de vida tornou-a particularmente sensível. As ferramentas atualmente ao dispor das famílias apresentam, do nosso ponto de vista, três limitações relevantes: as aplicações bancárias mostram movimentos individuais sem categorização ao nível familiar; as folhas de cálculo dependem de inserção manual e desatualizam-se com facilidade; e em ambos os casos não existe uma visão consolidada e simples do saldo real.

Optámos por desenvolver uma solução minimalista, em Python puro e sem dependências externas, que pudesse ser executada em qualquer máquina com um interpretador Python instalado. O foco foi a clareza do código e a aplicação rigorosa dos conceitos da unidade curricular, em detrimento de funcionalidades acessórias.

### 1.2. Objetivos do trabalho

O trabalho foi conduzido com os seguintes objetivos, alinhados com o enunciado:

- Aplicar de forma integrada os conhecimentos adquiridos na UC, em particular tipos de dados básicos, estruturas de controlo (if, for, while), listas, dicionários, funções com parâmetros e valor de retorno e modularização.
- Desenvolver um programa interativo, com menu em ciclo while, capaz de receber, validar e persistir dados introduzidos pelo utilizador.
- Implementar uma função obrigatória de cálculo (no caso, média mensal) e pelo menos duas funções auxiliares com ou sem retorno.
- Adotar boas práticas de programação: nomes descritivos, comentários explicativos, indentação consistente e separação clara entre interface e lógica de negócio.

## 2. Análise de Requisitos

Esta secção sintetiza os requisitos extraídos do enunciado e a forma como foram operacionalizados pelo grupo. Distinguem-se requisitos funcionais (o que o programa deve fazer) de requisitos não funcionais (como deve fazê-lo).

## 2.1. Requisitos funcionais

| Requisito do enunciado                      | Como foi cumprido                                                                                                                                   |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Menu interativo com ciclo while             | Implementado em main.py: ciclo while True com 8 opções e saída controlada pela opção 8 ("Guardar e sair").                                          |
| Inserção de dados via input()               | Inserção de receitas (opção 1) e despesas (opção 2) com leitura de descrição, valor e categoria.                                                    |
| Validação de entradas                       | Função validar_lancamento() em logic.py verifica id, descrição, valor positivo e categoria; conversão de valor protegida com try/except ValueError. |
| Armazenamento em listas de dicionários      | Receitas e despesas são guardadas em dados["receitas"] e dados["despesas"], cada uma como lista de dicionários (ver capítulo 4).                    |
| Função obrigatória com parâmetros e retorno | calcular_media_mensal(dados, ano, tipo) — devolve a média mensal de receitas ou despesas num ano, dividindo pelos meses com lançamentos.            |
| Pelo menos duas funções auxiliares          | Implementadas mais de dez funções auxiliares em logic.py (ver capítulo 5), entre cálculos, agregações e CRUD.                                       |

## 2.2. Requisitos não funcionais

- **Persistência:** os dados sobrevivem a reinícios do programa, sendo gravados em JSON com codificação UTF-8 e indentação para leitura humana.
- **Portabilidade:** a aplicação corre em Windows, Linux e macOS, sem dependências externas; usa apenas a biblioteca-padrão (json, os, re, datetime).
- **Robustez:** erros previsíveis (ficheiro inexistente, valor não numérico, ID inválido) são tratados sem terminar a execução de forma abrupta.
- **Legibilidade:** o código segue as convenções PEP 8, com nomes em português europeu, docstrings nas funções públicas e separação clara entre interface (main.py) e regras de negócio (logic.py).

## 2.3. Tema escolhido

De entre os temas sugeridos, o grupo escolheu Orçamento Familiar (despesas e receitas) por se tratar de um problema concreto, com vocabulário familiar a todos os elementos e com requisitos naturalmente compatíveis com listas de dicionários, validações numéricas e cálculos agregados. Permite ainda demonstrar competências adicionais úteis em contextos profissionais futuros, nomeadamente a manipulação de ficheiros JSON e a separação modular de responsabilidades.

## 3. Arquitetura da Solução

A aplicação está organizada em três artefactos principais, com responsabilidades disjuntas. Esta separação foi uma decisão de design deliberada: facilita a divisão de tarefas no grupo, permite testar a lógica de forma independente da interface e mantém o ficheiro de dados como única fonte da verdade.

### 3.1. Estrutura de diretórios

```
orcamento-familiar/
├── Src/
│   ├── main.py           # interface CLI (Alexandre)
│   ├── logic.py          # regras de negócio (Tiago)
│   └── expenses_revenues.json # base de dados (Gonçalo)
├── README.md
└── .gitignore
```

### 3.2. Fluxo de dados

O fluxo de dados é unidirecional e sempre mediado pelo módulo de lógica. O main.py nunca lê nem escreve diretamente no ficheiro JSON: todas as operações de leitura, escrita, validação e cálculo passam obrigatoriamente por logic.py. Esta regra simplifica a manutenção e elimina uma classe inteira de erros (estados inconsistentes entre o que está em memória e o que está em disco).

```
Utilizador → main.py → logic.py → expenses_revenues.json
(input)      (UI/menus)  (cálculos)      (persistência)
```

### 3.3. Responsabilidades por módulo

| Módulo                 | Responsabilidade                                                                                                                                                                        |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| main.py                | Apresenta o menu, lê input do utilizador, formata moeda em estilo português (1.500,00 €) e invoca as funções de logic.py. Não conhece o formato do ficheiro JSON.                       |
| logic.py               | Implementa toda a lógica de negócio: persistência, validação, geração de IDs, CRUD, filtros temporais, totais, saldo, média mensal e agrupamentos por categoria.                        |
| expenses_revenues.json | Base de dados em texto simples, contendo metadata, lista de membros do agregado, lista de receitas e lista de despesas. É criado automaticamente na primeira execução, caso não exista. |

## 4. Modelo de Dados

O ficheiro expenses\_revenues.json está organizado em quatro secções de topo: metadata, membros, receitas e despesas. Esta estrutura foi pensada para permitir, no futuro, a evolução natural da aplicação (nomeadamente o suporte a múltiplas famílias ou a agregação por membro) sem alterações disruptivas.

### 4.1. Esquema geral

```
{
  "metadata": {
    "familia": "Família Silva",
    "moeda": "EUR",
    "criado_em": "2026-02-01",
    "ultima_atualizacao": "2026-02-01"
  },
  "membros": [
    {
      "nome": "João",
      "idade": 35,
      "sexo": "M",
      "data_nascimento": "1991-03-15",
      "data_registro": "2026-02-01"
    },
    {
      "nome": "Maria",
      "idade": 33,
      "sexo": "F",
      "data_nascimento": "1993-07-22",
      "data_registro": "2026-02-01"
    },
    {
      "nome": "Pedro",
      "idade": 8,
      "sexo": "M",
      "data_nascimento": "2018-01-10",
      "data_registro": "2026-02-01"
    },
    {
      "nome": "Ana",
      "idade": 6,
      "sexo": "F",
      "data_nascimento": "2020-05-03",
      "data_registro": "2026-02-01"
    }
  ],
  "receitas": [
    {
      "descricao": "Salário de João",
      "valor": 1500,
      "data_registro": "2026-02-01"
    },
    {
      "descricao": "Salário de Maria",
      "valor": 1200,
      "data_registro": "2026-02-01"
    },
    {
      "descricao": "Pensão de João",
      "valor": 500,
      "data_registro": "2026-02-01"
    }
  ],
  "despesas": [
    {
      "descricao": "Aluguer",
      "valor": 500,
      "data_registro": "2026-02-01"
    },
    {
      "descricao": "Luz",
      "valor": 100,
      "data_registro": "2026-02-01"
    },
    {
      "descricao": "Água",
      "valor": 50,
      "data_registro": "2026-02-01"
    },
    {
      "descricao": "Internet",
      "valor": 30,
      "data_registro": "2026-02-01"
    },
    {
      "descricao": "Alimentação",
      "valor": 400,
      "data_registro": "2026-02-01"
    },
    {
      "descricao": "Transporte",
      "valor": 150,
      "data_registro": "2026-02-01"
    },
    {
      "descricao": "Saúde",
      "valor": 100,
      "data_registro": "2026-02-01"
    },
    {
      "descricao": "Educação",
      "valor": 200,
      "data_registro": "2026-02-01"
    },
    {
      "descricao": "Lazer",
      "valor": 100,
      "data_registro": "2026-02-01"
    }
  ]
}
```

```

"membros": [
  { "id": 1, "nome": "João Silva", "relacao": "pai" },
  { "id": 2, "nome": "Maria Silva", "relacao": "mãe" },
  { "id": 3, "nome": "Tomás Silva", "relacao": "filho" }
],
"receitas": [ { ... } ],
"despesas": [ { ... } ]
}

```

## 4.2. Estrutura de uma receita

```

{
  "id": "R001",
  "data": "2026-04-01",
  "descricao": "Salário Abril",
  "categoria": "salario",
  "membro_id": 1,
  "valor": 1450.00,
  "recorrente": true,
  "frequencia": "mensal"
}

```

## 4.3. Estrutura de uma despesa

```

{
  "id": "D001",
  "data": "2026-04-05",
  "descricao": "Renda da casa",
  "categoria": "habitacao",
  "subcategoria": "renda",
  "valor": 750.00,
  "metodo_pagamento": "transferencia",
  "recorrente": true,
  "frequencia": "mensal"
}

```

## 4.4. Categorias adaptadas à realidade portuguesa

As categorias foram escolhidas para refletir a estrutura típica de um orçamento doméstico em Portugal.

### Receitas

- salario, subsidio\_refeicao, subsidio\_ferias, subsidio\_natal, prestacoes\_sociais, extras

### Despesas

- habitacao, utilidades, alimentacao, transportes, saude, educacao, lazer, vestuario, outros

## 4.5. Decisões de design

- IDs sequenciais com prefixo (R001, D001, ...): permitem distinguir o tipo do lançamento sem inspecionar o resto do registo e oferecem um identificador estável para operações de edição e remoção.
- Datas em ISO 8601 (YYYY-MM-DD): facilitam ordenação lexicográfica e filtragem por mês/ano através de simples startswith.
- Valores em float arredondados a duas casas: suficiente para um orçamento doméstico e evita a complexidade do tipo Decimal nesta fase do curso.
- Auto-completar campos opcionais: o módulo logic.py preenche valores por omissão (data de hoje, frequência "unica", método de pagamento "dinheiro") quando o main.py cria um lançamento mais simples, mantendo o ficheiro JSON sempre válido.

## 5. Implementação

Esta secção detalha as funções desenvolvidas, agrupadas por responsabilidade, e destaca os pontos do código que cumprem requisitos específicos do enunciado.

### 5.1. API do módulo logic.py

| Função                                   | Devolve | Descrição                                                                          |
|------------------------------------------|---------|------------------------------------------------------------------------------------|
| carregar_dados(caminho)                  | dict    | Lê o JSON do disco; se o ficheiro não existir, devolve uma estrutura vazia válida. |
| guardar_dados(dados, caminho)            | None    | Grava o JSON em disco (UTF-8, indent=2) e atualiza a metadata.                     |
| adicionar_receita(dados, receita)        | dict    | Insere uma nova receita, auto-completando campos opcionais.                        |
| adicionar_despesa(dados, despesa)        | dict    | Insere uma nova despesa, auto-completando campos opcionais.                        |
| remover_lancamento(dados, id_lanc)       | dict    | Remove o lançamento com o ID indicado (procura em receitas e despesas).            |
| editar_lancamento(dados, id, alteracoes) | dict    | Edita campos arbitrários de um lançamento existente.                               |
| total_receitas(dados, mes, ano)          | float   | Soma das receitas, opcionalmente filtrada por mês/ano.                             |
| total_despesas(dados, mes, ano)          | float   | Soma das despesas, opcionalmente filtrada por mês/ano.                             |
| saldo(dados, mes, ano)                   | float   | Receitas menos despesas no período indicado.                                       |
| calcular_media_mensal(dados, ano, tipo)  | float   | Média mensal de receitas ou despesas num ano (função obrigatória do enunciado).    |
| agrupar_por_categoria(dados, tipo)       | dict    | Total acumulado por categoria, para receitas ou despesas.                          |

| Função                                      | Devolve | Descrição                                                |
|---------------------------------------------|---------|----------------------------------------------------------|
| <code>resumo_mensal(dados, mes, ano)</code> | dict    | Resumo completo de um mês: totais, saldo e agrupamentos. |
| <code>validar_lancamento(lancamento)</code> | bool    | Valida campos obrigatórios e formato.                    |
| <code>gerar_id(tipo, dados)</code>          | str     | Gera o próximo ID sequencial (R001, D012, ...).          |

## 5.2. Função obrigatória — `calcular_media_mensal`

O enunciado exige uma função com parâmetros e valor de retorno, com assinatura equivalente a `calcular_media(lista_numeros)`. Optou-se por uma variante mais expressiva, que recebe a estrutura completa de dados, o ano e o tipo (receitas ou despesas), e devolve a média mensal apenas sobre os meses com lançamentos:

```
def calcular_media_mensal(dados, ano, tipo="despesas"):
    if tipo == "receitas":
        lista = dados.get("receitas", [])
    else:
        lista = dados.get("despesas", [])

    totais_por_mes = {}
    for item in lista:
        data = item.get("data", "")
        if not data.startswith(str(ano)):
            continue
        mes = data[5:7]
        if mes in totais_por_mes:
            totais_por_mes[mes] += item.get("valor", 0)
        else:
            totais_por_mes[mes] = item.get("valor", 0)

    if len(totais_por_mes) == 0:
        return 0.0

    soma = 0.0
    for mes in totais_por_mes:
        soma += totais_por_mes[mes]
    return round(soma / len(totais_por_mes), 2)
```

A decisão de dividir pelo número de meses com lançamentos (em vez de fixar 12) foi tomada deliberadamente: para um histórico de três meses, dividir por 12 produziria uma média artificialmente baixa e enganadora. Esta opção é fiel ao espírito da estatística descritiva — calcular a média sobre os dados existentes, não sobre os meses possíveis.



### 5.3. Interface — main.py

A interface é totalmente em texto e segue um padrão clássico de menu em loop. O ciclo while True só termina quando o utilizador escolhe a opção 8 ("Guardar e sair"), garantindo que os dados não são perdidos por engano. Após cada operação, o ecrã é limpo (cls em Windows, clear em Linux/macOS) para manter a interface arrumada.

```
=====
      ORÇAMENTO FAMILIAR
=====
1. Adicionar Receita
2. Adicionar Despesa
3. Listar Receitas
4. Listar Despesas
5. Saldo do Mês
6. Resumo por Categoria
7. Editar/Remover Lançamento
8. Guardar e Sair
=====
Escolha uma opção:
```

A formatação dos valores monetários respeita a convenção portuguesa, com ponto como separador de milhares e vírgula como separador decimal (ex.: 1.500,00 €). A função `formatar_moeda()` em `main.py` implementa esta transformação a partir do formato anglo-saxónico nativo do Python.

## 6. Validações e Tratamento de Erros

As validações estão concentradas em `logic.py`, na função `validar_lancamento()`, de modo a que qualquer ponto futuro de entrada de dados (não só o menu interativo, mas também eventuais importações em lote) beneficie das mesmas regras.

### 6.1. Regras aplicadas

- O lançamento tem de ser um dicionário.
- O id é obrigatório e tem de seguir o padrão `R\d{3,}` ou `D\d{3,}`, validado por expressão regular.
- A descrição tem de ser uma string não vazia (ignorando espaços).
- O valor tem de ser numérico (int ou float, excluindo bool) e estritamente positivo.
- A categoria tem de ser uma string não vazia.

### 6.2. Tratamento de erros na interface

O `main.py` usa um bloco `try/except` em torno da conversão do valor introduzido pelo utilizador. Se a conversão falhar (por exemplo, se o utilizador escrever "abc"), é apresentada uma mensagem amigável e o programa volta ao menu, sem terminar.

```
try:
    descricao = input("Descrição: ")
```

```

    valor = float(input("Valor: "))
    ...
except ValueError:
    print("⚠ Erro: Por favor, insira um número válido para o valor.")

```

De forma análoga, a função `carregar_dados()` em `logic.py` protege o programa contra a ausência do ficheiro JSON na primeira execução, devolvendo uma estrutura vazia válida em vez de lançar uma exceção. Esta abordagem permite que o programa funcione "out of the box" sem qualquer configuração inicial.

## 7. Execução e Demonstração

A aplicação não requer instalação de qualquer dependência externa. Para a executar, basta um interpretador Python 3.10 ou superior.

### 7.1. Comando de execução

```

# A partir da raiz do projeto
python Src/main.py

# Ou, equivalentemente
cd Src && python main.py

```

A variável `DB_PATH` em `main.py` é resolvida em relação à localização do próprio script, pelo que a aplicação funciona independentemente do diretório de trabalho a partir do qual é invocada.

### 7.2. Cenário de demonstração

Para a defesa do trabalho, prepará-mos um cenário ilustrativo com três meses de dados (fevereiro a abril de 2026), incluindo salários recorrentes, subsídio de refeição, despesas de habitação, alimentação e lazer, e um par de despesas extraordinárias. Este cenário permite mostrar:

- Listagem completa de receitas e despesas.
- Cálculo de saldo mensal isolado (ex.: abril de 2026).
- Resumo por categoria, evidenciando o peso da habitação no orçamento.
- Cálculo de média mensal de despesas no ano de 2026.
- Edição e remoção de lançamentos, com reflexo imediato no ficheiro JSON após a opção 8.

## 8. Distribuição de Tarefas

Para evitar conflitos no controlo de versões e tornar o trabalho avaliável de forma justa, atribui-se a cada elemento do grupo um ficheiro principal e uma branch dedicada. Foi acordado, antes do início do trabalho, que cada elemento só altera o seu ficheiro; alterações cruzadas só são feitas via pull request, após reunião curta.

| Elemento     | Ficheiro principal     | Responsabilidade                                                                                                              |
|--------------|------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Gonçalo F.   | expenses_revenues.json | Modelagem da estrutura de dados; categorias adaptadas a Portugal; dataset de demonstração; documentação do esquema no README. |
| Tiago F.     | logic.py               | Implementação das funções de persistência, validação, CRUD, filtros, totais, saldo, média mensal e agrupamentos.              |
| Alexandre T. | main.py                | Interface CLI: menu, leitura de input, formatação portuguesa de moeda, integração com logic.py.                               |

## 8.1. Convenções de equipa

- Branches dedicadas: feature/main, feature/logic, feature/json.
- Mensagens de commit no formato Conventional Commits (feat:, fix:, docs:, chore:).
- Sincronização diária no início do dia: git pull origin dev na branch pessoal.
- Reunião curta antes do merge final, para alinhar tipos de campos e nomes de funções partilhadas.

## 9. Reflexão Crítica e Trabalho Futuro

### 9.1. O que correu bem

A separação rígida em três módulos, decidida no primeiro dia, revelou-se a melhor decisão do projeto. Permitiu trabalhar em paralelo sem conflitos significativos no Git e tornou cada peça testável de forma isolada. O facto de o JSON ser criado automaticamente na primeira execução eliminou também uma fonte habitual de erros ("o ficheiro não existe").

A escolha de listas de dicionários como estrutura central, em vez de classes ou tipos compostos, manteve o código alinhado com a matéria da unidade curricular e mostrou-se suficientemente expressiva para todos os requisitos.

### 9.2. Limitações conscientes

- Os valores são guardados como float, com arredondamento a duas casas. Para uma aplicação de produção, far-se-ia uso de Decimal para evitar erros subtis de vírgula flutuante em somas longas.
- Não existe controlo de concorrência: dois utilizadores em simultâneo no mesmo ficheiro provocariam perda de dados. No contexto familiar para o qual a aplicação foi pensada, o risco é negligenciável.
- A interface é em texto. Para utilizadores menos familiarizados com terminais, uma interface gráfica simples (em tkinter, por exemplo) seria mais acolhedora.

### 9.3. Possíveis evoluções

- Suporte a múltiplas famílias num único ficheiro, com login simples por nome.

- Exportação de relatórios em CSV ou PDF, para arquivo mensal.
- Gráficos no terminal (com bibliotecas como rich) ou interface gráfica em tkinter.
- Definição de orçamentos por categoria, com alertas quando o consumo se aproxima do limite.
- Recorrência automática: criar lançamentos repetidos com base no campo frequência já presente no esquema.

## 9.4. Aprendizagens

Para além do domínio das estruturas de controlo e funções, o projeto foi a primeira oportunidade de aplicar, num contexto académico, conceitos que extravasam o programa da UC: divisão modular, controlo de versões em equipa, design de uma estrutura de dados pensada para evoluir e separação entre interface e regras de negócio. São competências transferíveis para qualquer projeto futuro, dentro ou fora do meio académico.

## 10. Conclusão

O trabalho desenvolvido cumpre integralmente os requisitos definidos no enunciado: dispõe de menu interativo em ciclo while, permite a inserção e armazenamento de dados em listas de dicionários, valida as entradas, persiste em ficheiro de texto estruturado e implementa a função obrigatória de cálculo com parâmetros e retorno, juntamente com mais de uma dezena de funções auxiliares.

Mais relevante do que o cumprimento estrito dos requisitos, foi a oportunidade de exercitar boas práticas de desenvolvimento — modularização, separação de responsabilidades, validações centralizadas, controlo de versões em equipa e documentação clara — que extravasam o âmbito da unidade curricular e que constituirão, esperamos, a base para projetos futuros mais ambiciosos.

Agradecemos ao professor Mário Amorim a flexibilidade na escolha do tema e o acompanhamento ao longo do semestre.

## Referências Bibliográficas

As referências seguintes foram consultadas durante o desenvolvimento do trabalho, sendo as três primeiras as principais fontes da unidade curricular.

- Seixas, J. A. (2021). Introdução à Programação em Python. Lisboa: FCA — Editora de Informática.
- Downey, A. B. (2015). Think Python: How to Think Like a Computer Scientist (2.<sup>a</sup> ed.). Green Tea Press. Disponível em <https://greenteapress.com/wp/think-python-2e/>
- Zelle, J. M. (2017). Python Programming: An Introduction to Computer Science (3.<sup>a</sup> ed.). Wilsonville, OR: Franklin, Beedle & Associates.
- Lutz, M. (2013). Learning Python (5.<sup>a</sup> ed.). Sebastopol, CA: O'Reilly Media.

- Python Software Foundation. (2026). Python 3 Documentation. Disponível em <https://docs.python.org/3/>
- W3Schools. (2026). Python Tutorial. Disponível em <https://www.w3schools.com/python/>
- Amorim, M. (2026). Programação Estruturada e Tipos de Dados — Ficha de Unidade Curricular e materiais de apoio. ISTECS Porto, ano letivo 2025/2026.